
Chapter 18 Advanced Data Structures

18.1 Hashing

- [Array Set Implementations !\[\]\(c8dce68b26731c7aa5915072fc9d68dd_img.jpg\)](#)
- [Hash Functions and Hash Tables !\[\]\(76b3245de86167eba9fcdc9cc9f32aa4_img.jpg\)](#)
- [Collisions !\[\]\(13db7587f50867332e5bedc6a161739d_img.jpg\)](#)
- [Rehashing !\[\]\(7be5ea91065783fbb69e41ba5d9680f7_img.jpg\)](#)
- [Hashing Non-Integer Data !\[\]\(20b6116a35a537c491fe1e2cc04e020e_img.jpg\)](#)
- [Hash Map Implementation !\[\]\(9e6cd34ccb2e621bcc854e8b124ba455_img.jpg\)](#)

18.2 Priority Queues and Heaps

- [Priority Queues !\[\]\(f15d3c54be60b4fd0ce1da9fb3f67256_img.jpg\)](#)
- [Introduction to Heaps !\[\]\(7bf135d42c40a6430c927b2fd03d7659_img.jpg\)](#)
- [Removing from a Heap !\[\]\(2bcc37677ea6b96900e4d746ad300082_img.jpg\)](#)
- [Adding to a Heap !\[\]\(b62812e390f75b509ead0f847e76b4ce_img.jpg\)](#)
- [Array Heap Implementation !\[\]\(702f396a3c354a80d179cf62e75a5343_img.jpg\)](#)

Introduction

In this chapter, we will explore the implementation of two powerful data structures called hash tables and heaps. Both of these structures are used in Java's Collections Framework to implement collections such as `HashSet`, `HashMap`, and `PriorityQueue`. These clever data structures allow for efficient operations such as adding, removing, searching, and ordering elements.

Upon completion of this chapter you will have seen all of the major implementation strategies used in the primary collections in the Java Collections Framework: arrays (`ArrayList`, `Stack`), linked lists (`LinkedList`), binary trees (`TreeSet`, `TreeMap`), hash tables (`HashSet`, `HashMap`), and heaps (`PriorityQueue`).

18.1 Hashing

Hashing is a technique for efficiently mapping data elements to indexes in an array so that they can be added, removed, and searched very quickly. Hashing is the underlying technique used to implement the `HashSet` and `HashMap` classes. Hashing makes it possible to have a collection that can add, remove, and search for data elements in a constant amount of time (also called $O(1)$), as discussed in [Chapter 13](#) , making it extremely fast for many common use cases.

Array Set Implementations

Suppose we want to store a set of integers using an array as the underlying data structure and that the most common operations we want to enable are insertion (`add`), deletion (`remove`), and testing for membership (`contains`). Further suppose that the order in which the elements are stored does not matter, so long as we can implement all of these operations efficiently. The key concern is that the structure should be fast for these three operations.

We could follow the same general idea as `ArrayIntList`, using an unfilled array and a `size` field to represent the data in the set:

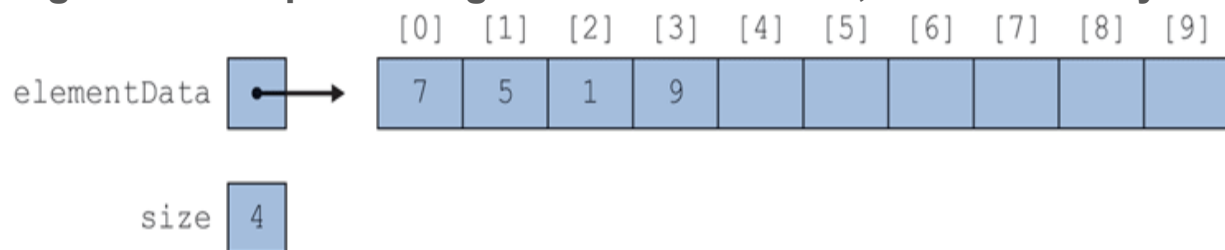
```

public class ArrayIntSet {
    private int[] elementData;
    private int size;
    |
    // Constructs an empty set.
    public ArrayIntSet() {
        elementData = new int[10];
        size = 0;
    }
    ...
}

```

As each element is inserted, it is stored at the next available index and the `size` field is incremented. So if the array's length is 10 and the client adds the values 7, 5, 1, and 9, the set would have the internal state shown in [Figure 18.1](#). The blank array cells actually contain 0s, but they are unimportant because they are past the size.

Figure 18.1 Representing a set as an unfilled, unsorted array



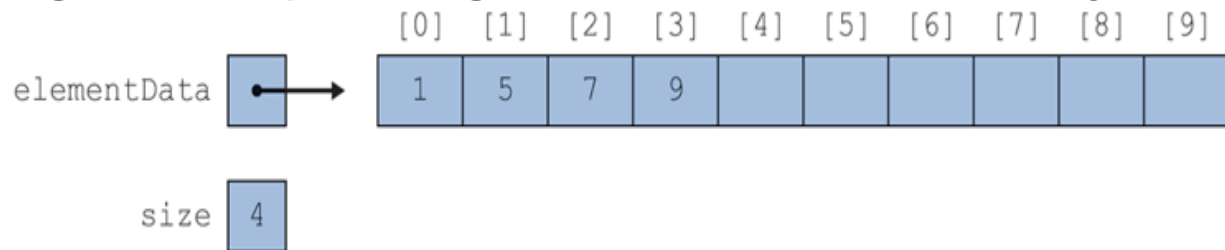
Is this a good way to implement a set? Think about how the common operations would be written. Adding an element is efficient ($O(1)$); simply place it in the next free slot and increment the `size` field.

Removal is an inefficient linear search through the array for the value to remove. Once the value is found, since order is unimportant, you can quickly remove it by replacing it by the last element in the array, rather than having to shift any following elements left by one index; but the search to find it is still slow. We might have to examine all of the elements (on average, about half of them), so removal is $O(N)$. Testing for membership is also slow, a linear search over all elements to try to find the value of interest. To figure out whether the set contains the value 9, we must look at the 7, 5, 1, and 9 before it is found.

Removal and searching from the unsorted array are slow because there's no particular way to guess where the element value will be found, if it is in the array at all. But with some creativity we can improve upon this situation. We've seen some strategies that take advantage of sorted data, such as the binary search algorithm. Binary search does not need to examine every index of the array to find an element; at each step the algorithm eliminates half of the remaining search space, resulting in a $O(\log N)$ runtime.

We could base our set on a sorted array, as shown in [Figure 18.2](#). As elements are added to this sorted array, they must be placed into the proper indexes to maintain sorted order. The implementation of insertion or removal requires two steps: binary searching for the right index for the element, then shifting any following elements left or right. For example, if the value 2 is added, the 5, 7, and 9 must be shifted right by one index to make room for it.

Figure 18.2 Representing a set as an unfilled, sorted array



The sorted array structure would be faster to search because we could implement our `contains` method as a binary search. Adding and removing would be $O(N)$ in the worst case because of the shifting. Still, this is arguably a better implementation than the unsorted array if the primary concern is the speed of the `contains` operation.

Hash Functions and Hash Tables

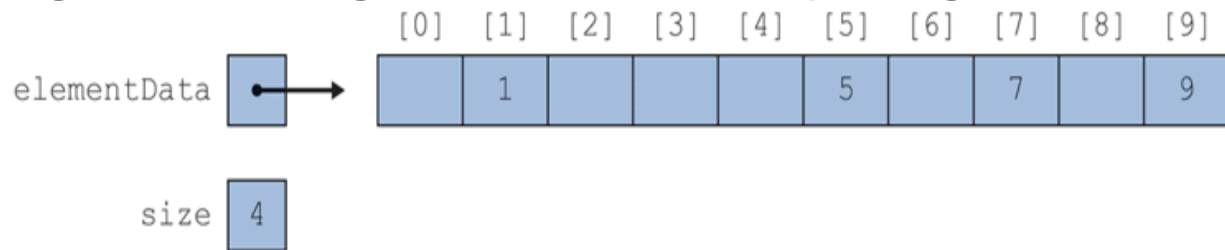


Implementing a set using a sorted array is a step up from an unsorted one, but it's possible to do better. Let's explore how to use

a special mapping strategy called a hash function to organize the elements of our set and search it quickly.

Recall that the order of the elements doesn't matter, as long as the set can add, remove, and find elements. The elements can be placed anywhere, as long as they can be found later. Here's an odd but powerful idea: What if we stored element value k at index k ? For example, if you tell the set to add the value 5, store it at index 5. If we used this technique, the set storing 7, 5, 1, and 9 would have the structure shown in [Figure 18.3](#).

Figure 18.3 Storing set elements at corresponding indexes



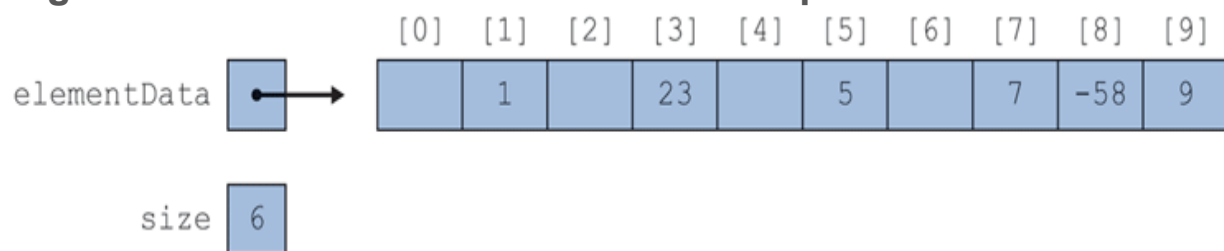
If we stored our set data using this technique, the three basic set operations become extremely efficient to implement. Inserting a value k simply involves going to index k and storing k there; no searching or shifting is needed. Removing the value k requires going to index k and changing the value back to 0. Testing for membership of a value k (`contains`) requires looking at index k to see whether k is stored there; if so, the value is part of the set, and if not, it isn't.

You may already be thinking of some problems with this implementation strategy. What if the client tries to add a value that is outside the range 0 to 9, such as 23? We could start with a larger array; if `elementData` has length 100, our 23 will fit nicely. But no matter how large the array becomes, the client can always add a larger integer outside its bounds. Negative numbers also pose a problem because arrays never have negative indexes.

To get around these issues, let's patch our storage technique. Instead of always storing element value k at index k , we'll limit k 's value by modding it by the array capacity. So if the array length is 10, the value 23 would be inserted at index 3 because $23 \% 10$ equals 3. To fix negative numbers, we'll take the absolute value of k and apply the same technique. So the value -58 would be inserted at index 8.

Figure 18.4 shows what the array would look like after the insertion of 23 and -58 using our new mod technique. The set still has the same speedy runtime for the common operations because each element still has a single index into which it will be placed.

Figure 18.4 Use of hash function with wrap-around



Since our mapping from element value to preferred index now has a bit of complexity to it, we might turn it into a method that accepts the element value as a parameter and returns the right index for that value. Such a method is referred to as a *hash function*, and an array that uses such a function to govern insertion and deletion of its elements is called a *hash table*. The individual indexes in the hash table are also sometimes informally called *buckets*.

Our hash function so far is the following:

```
private int hashFunction(int value) {  
    return Math.abs(value) % elementData.length;  
}
```

Hash Function

A method for rapidly mapping between element values and preferred array indexes at which to store those values.

Hash Table

An array that stores its elements in indexes produced by a hash function.

Collisions

There is still a problem with our current hash table. Because our hash function wraps values to fit in the array bounds, it is now possible that two values could have the same preferred index. For example, if we try to insert `45` into the hash table, it maps to index `5`, conflicting with the existing value `5`. This is called a *collision*. Our implementation is incomplete until we have a way of dealing with collisions. If the client tells the set to insert `45`, the value `45` must be added to the set somewhere; it's up to us to decide where to put it.

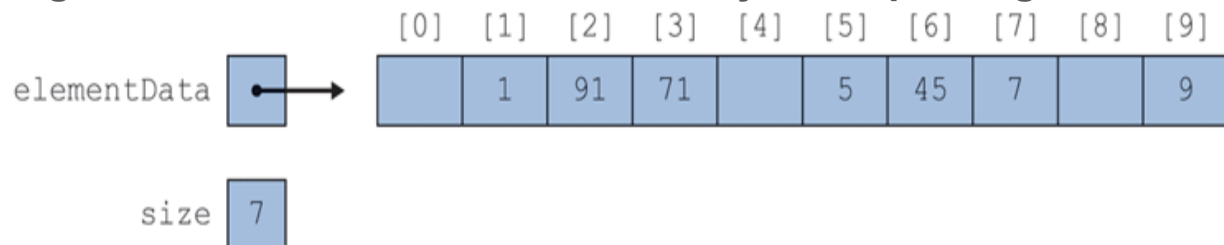
Collision

When two or more element values in a hash table produce the same result from its hash function, indicating that they both prefer to be stored in the same index of the table.

One common way of resolving collisions is called *probing*, which involves looking for another index to use if the preferred index is taken. For example, if the client wants to add 45 and index 5 is in use, we could just put 45 at the next available index, which is 6 in our example. (Looking forward one index at a time for the next free index is called *linear probing*, but there are other kinds of probing such as *quadratic probing*, which involves jumping around to various places in the hash table.)

Figure 18.5 shows the hash table's state if the values 45, 91, and 71 are added and linear probing is used to resolve the collisions. The 45 conflicts with 5 and is put into index 6. The 91 conflicts with existing value 1 and is put into index 2. The 71 conflicts with 1 and 91 and must be put into index 3.

Figure 18.5 Hash collisions resolved by linear probing



Probing gets around the collision problem, but it introduces new problems of its own. For one, it is no longer as simple to find an element's value. A value whose hash function evaluates to k might be stored at index k , but if something other than k is there, it might

be at index $k + 1$, $k + 2$, etc. So we would need to patch our other methods such as `contains` accordingly. We can make the appropriate modifications, but we also have to think about the original goals of this implementation. Searching for elements is supposed to be fast, and if we have to probe through a lot of elements to find anything, we're losing the efficiency we sought after in the first place.

Probing

Resolving hash collisions by placing elements at other indexes in the table rather than their preferred indexes.

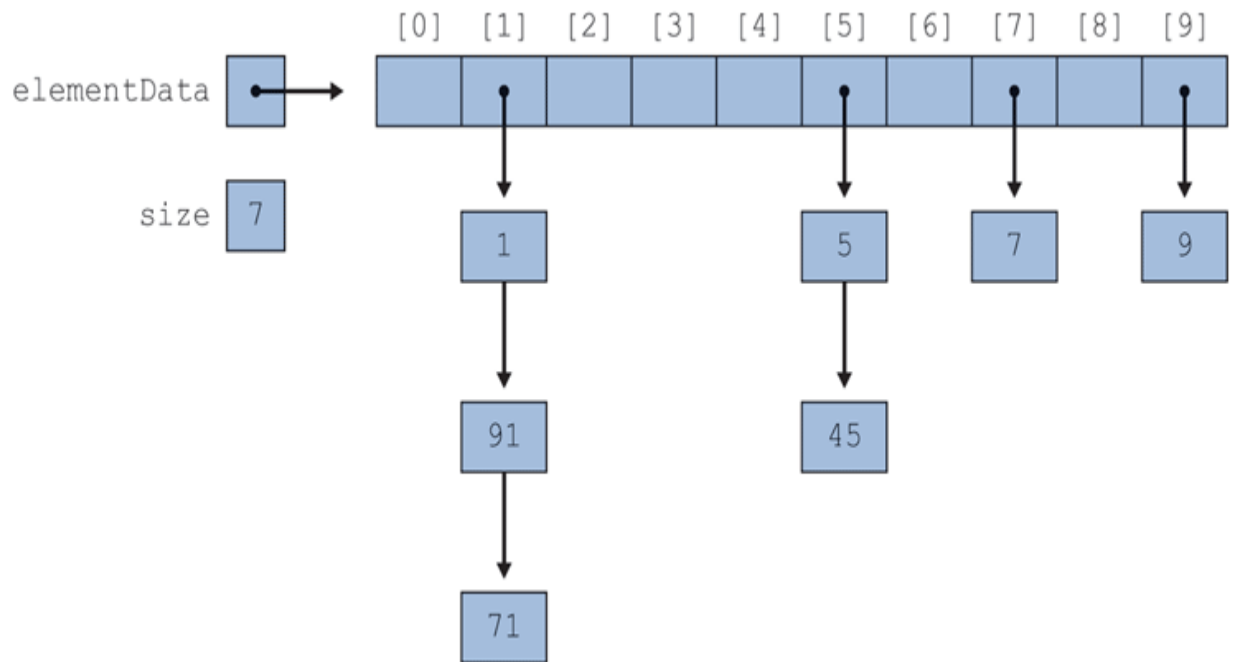
Another problem is that the hash table can get full, resulting in no free slots to store a value. The example array of size 10 can hold only 10 elements. If the client tries to add an eleventh element, the array must be resized. Resizing a hash table is a nontrivial operation that we'll discuss in the next section.

Even if the array is not entirely full, if many elements are next to each other it can slow down the runtime for later operations. For example, to add the value `25` to the hash table in [Figure 18.5](#)  requires looking at four buckets: indexes 5, 6, 7, and 8 (where the value is finally placed). After doing so, a search for the value `95`

(which is not found in the table) would need to examine indexes 5, 6, 7, 8, 9, and 0 before finally giving up. When elements clump up near each other like this, it is called *clustering*, and it is desirable to have as little clustering as possible in a hash table for it to perform efficiently.

Another way of dealing with the collision problem is to change our internal data structure. Collisions would not be a problem if each array index could store more than one value. This is possible if we change the array to be an array of lists rather than an array of integers. The list at index k will store all elements that the hash function maps to k . **Figure 18.6**  demonstrates this idea. To add an element to the hash table, we go to its preferred index and add the element value to the list stored at that index. Resolving collisions by storing hash elements in lists is called *separate chaining*. The real `HashSet` and `HashMap` provided in `java.util` use separate chaining internally to resolve collisions.

Figure 18.6 Hash collisions resolved by separate chaining



Separate Chaining

Resolving hash collisions by having each index of the table store a list of values rather than a single value.

As with probing, separate chaining faces an efficiency issue if the size of the set becomes large enough, especially if the numbers happen to have a lot of collisions with each other. For example, if the client adds a lot of numbers to the set that end with 1, there will be a long list of elements at index 1 in the hash table, and we'll have to perform a long search through the list to find values. One way to mitigate this issue is to use a larger array to store the element data,

which will result in fewer collisions and shorter lists at each index. Another good idea is to use an array size that is not a multiple of 10 or any other round number, to avoid number patterns that are likely to collide. Many hash table implementations use a prime number for the table capacity.

We'll implement the lists of elements as lists of node objects, similar to the implementation of linked lists in [Chapter 16](#) . An inner class `HashEntry` represents a single node in such a list:

```
// Represents a single value in a chain stored in one hash
bucket.
private class HashEntry {
    private int data;
    private HashEntry next;
    public HashEntry(int data) {
        this(data, null);
    }
    public HashEntry(int data, HashEntry next) {
        this.data = data;
        this.next = next;
    }
}
```

Using this inner class we can now implement the `add` and `contains` operations on our set. The `add` method adds a new entry to the table if the element is not already found in the table. Before adding an

element to the set, we make sure it is not a duplicate by calling `contains`. The `contains` method looks through the list at the appropriate hash bucket index to see if that value is found in the list. When elements are added, we insert them at the front of the appropriate linked list. This is faster than traversing the links all the way to the end of the list to insert the element.

```
// Adds the given element to this set, if it was not
// already contained in the set.
public void add(int value) {
    if (!contains(value)) {
        // insert new value at front of list
        int bucket = hashFunction(value);
        elementData[bucket] = new HashEntry(value,
elementData[bucket]);
        size++;
    }
}

// Returns true if the given value is found in this set.
public boolean contains(int value) {
    int bucket = hashFunction(value);
    HashEntry current = elementData[bucket];
    while (current != null) {
        if (current.data == value) {
            return true;
        }
    }
}
```

```
        current = current.next;
    }
    return false;
}
```

We can also implement the `remove` method now. Removal is a bit trickier than adding, because we have to make sure to handle all of the possible cases. If the element to be removed is at the front of its linked list, we must adjust the front reference; otherwise we must change the `next` reference of some existing node in the list. As always with linked lists, we must be careful to check for `null` and not try to traverse past the end of a list.

```
// Removes the given value if it is contained in the set.
public void remove(int value) {
    int bucket = hashFunction(value);
    if (elementData[bucket] != null) {
        // check front of list
        if (elementData[bucket].data == value) {
            elementData[bucket] = elementData[bucket].next;
            size--;
        } else {
            // check rest of list
            HashEntry current = elementData[bucket];
            while (current.next != null && current.next.data !=
value) {
                current = current.next;
            }
        }
    }
}
```

```

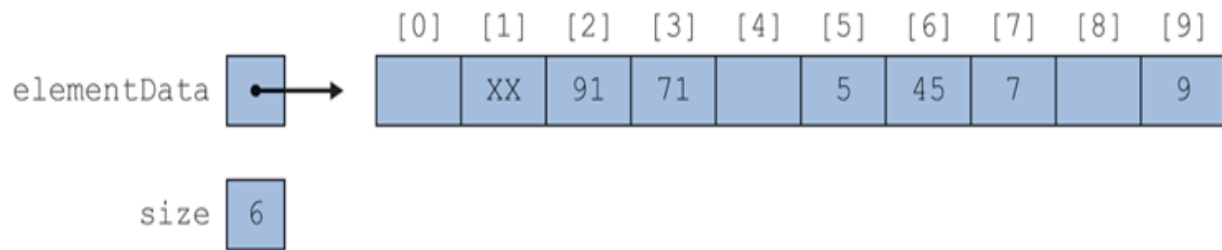
    }
}

// if the element is found, remove it
if (current.next != null) {
    current.next = current.next.next;
    size--;
}
}
}
}
}

```

Removing from a hash table that uses probing has different complications. If we start with the hash table from [Figure 18.5](#) and decide to remove the value 1, we can't just replace it with a blank value, because there is a chain of other elements (91 and 71) that probed to the following indexes 2 and 3. If a later `contains(71)` call is made on the set, the code might mistakenly think that bucket 1 is empty and therefore incorrectly decide that 71 is not contained in the set. What is often done instead is to replace the removed value with a special marker value to indicate that a removal occurred. That way when `contains` or `add` is called later, it can recognize that value and realize that other values might follow the removed bucket. Flagging buckets with this special marker is more efficient than trying to shift values back in the hash table. [Figure 18.7](#) shows an example removal of the value 1.

Figure 18.7 Removal of an element when probing is used



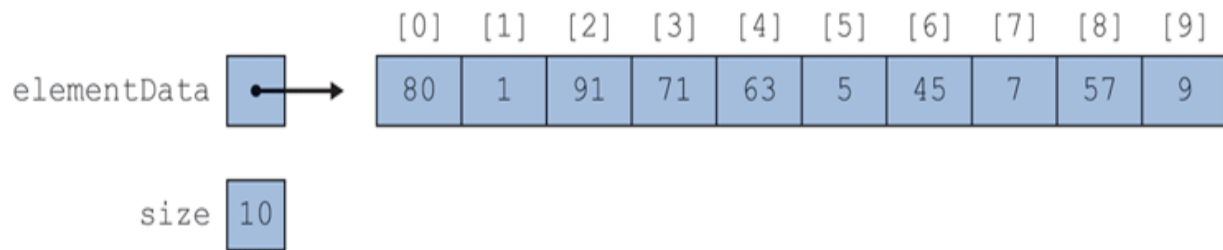
We will focus on our implementation that uses separate chaining rather than showing the code for the probing version. You can find both versions on our web site at <http://buildingjavaprograms.com>.

There are other ways of resolving collisions. For example, if the hash function results in a conflict, a secondary hash function can be used to find a new hash bucket. This is sometimes called *double hashing*. Double hashing and other methods of collision resolution are outside the scope of this chapter.

Rehashing

When a hash table becomes too full, it can be enlarged to give it additional capacity for storing elements. Consider the full hash table in **Figure 18.8** that uses linear probing to resolve collisions. In addition to the elements previously added, the client has inserted 80, 63, and 57, causing a completely full hash table. When the client tries to add an eleventh value, the hash table must be enlarged to make room for it.

Figure 18.8 Full hash table with linear probing



Resizing a hash table is not as simple as creating a larger array and copying each element over into it. The reason is because the element values may not have the same hash function result in the new larger array. For example, the value 91 maps to index 1 in the array of length 10 because $91 \% 10$ equals 1. But the same value 91 maps to index 11 in an array of length 20 because $91 \% 20$ equals 11. Therefore, when resizing a hash table array, it is important to reprocess each element using the hash function and place it into the new appropriate index. This process is called *rehashing*.

Suppose the client tries to add the value 35 to the full hash table.

Figure 18.9 shows a properly resized and rehashed table. Notice that 91, 71, 35, and 57 now sit in the second half of the array because their hash function results fall in that range. The value 63 also gets to sit in its preferred index of 3 because the element 71 that previously collided with it has moved to index 12. The rehashing also helps to decrease clustering; now the longest chain of consecutively occupied buckets is three.

Figure 18.9 Enlarged hash table after rehashing

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
80	1		63		5	45	7		9		91	71			35		57		

Rehashing

Resizing a hash table to increase its capacity and enabling it to store more elements, or store them more efficiently.

Hash tables that use separate chaining can also be rehashed, but the issues are slightly different. Technically a chained hash table never becomes completely “full” in the sense of becoming unable to add any more elements, because a new element can always be added to the list at a given index. But if the lists at each index become too long, the set operations become too slow. So it is still important to enlarge the hash table as its size grows. **Figure 18.10**  shows a chained table with the same elements as in **Figure 18.8** .

Figure 18.10 “Full” hash table with separate chaining

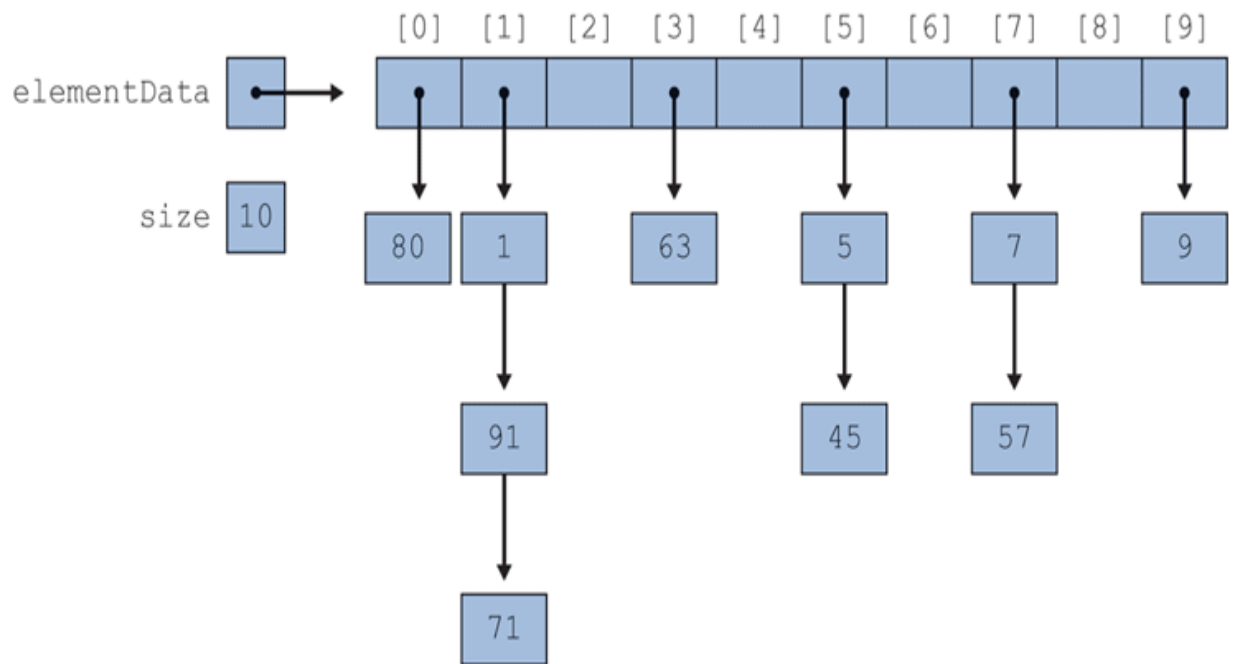
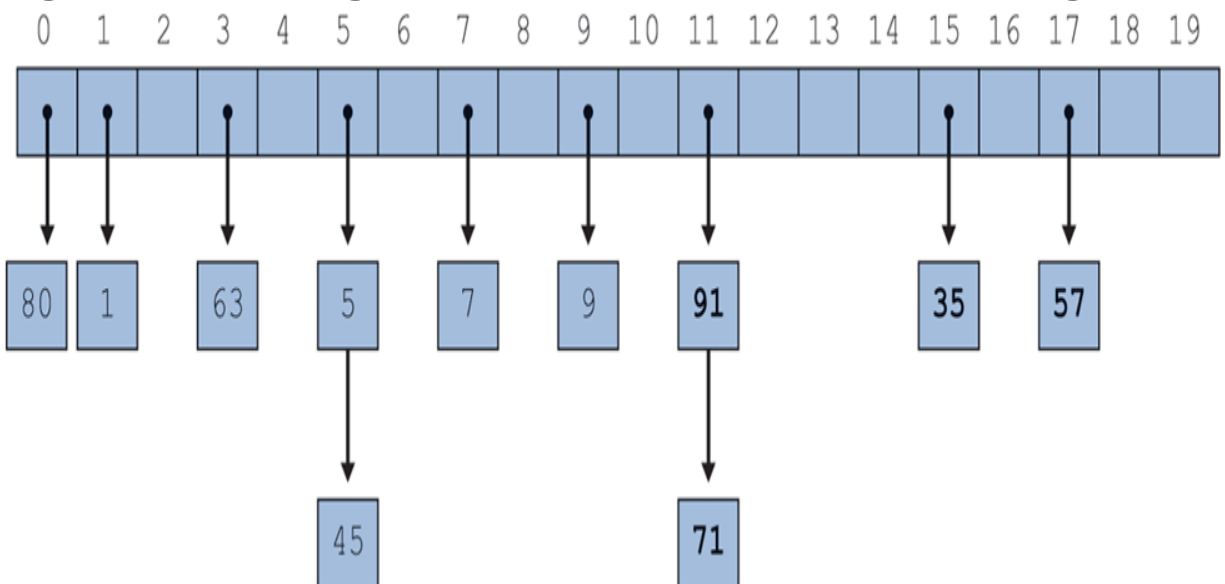


Figure 18.11 shows the same hash table after a rehashing to length 20 and the insertion of value 35. The values 91, 71, 35, and 57 are now stored in lists in the second half of the table.

Figure 18.11 Enlarged chained hash table after rehashing



In our examples we chose to resize the hash table once its size exceeded its capacity. But most hash tables do not wait so long to rehash, because a nearly or entirely full table starts to exhibit poor performance. A better alternative is to monitor the table's size relative to the array capacity, also called the *load factor* of the table, and resize once this ratio exceeds some threshold. The load factor is fairly easy to compute:

```
private double loadFactor() {  
    return (double) size / elementData.length;  
}
```

The real Java `HashSet` uses a default load factor of 0.75; in other words, once the number of elements is three fourths as large as the capacity, the table is enlarged. We can incorporate this logic into our own hash set with a constant and a check at the start of the `add` method:

```
public class HashIntSet {  
    private static final double MAX_LOAD_FACTOR = 0.75;  
    private HashEntry[] elementData;  
    private int size;  
    ...  
    public void add(int value) {  
        if (!contains(value)) {
```

```

        if (loadFactor() >= MAX_LOAD_FACTOR) {
            rehash();
        }
        ...

```

The simplest way to implement the actual rehashing operation is to replace the set's internal hash table with a new larger one, then loop over the old table and read all of its contents back into the new hash table. We can call the set's own `add` method to reinsert all of the data to avoid duplicating code:

```

private void rehash() {
    // replace element data array with a larger empty version
    HashEntry[] oldElementData = elementData;
    elementData = new HashEntry[2 * oldElementData.length];
    size = 0;

    // re-add all of the old data into the new array
    for (int i = 0; i < oldElementData.length; i++) {
        HashEntry current = oldElementData[i];
        while (current != null) {
            add(current.data);
            current = current.next;
        }
    }
}

```

Hashing Non-Integer Data

Our hash set works well for storing integer data, but we have not yet discussed how to apply hashing techniques to non-integer data. The key idea is still the same: We need a way of mapping from values to preferred indexes at which to store them. It is possible to come up with hash functions for other kinds of data that convert them into integers. For example, we could convert a string into an integer by adding the integer ASCII values of its characters in some way.

The idea of hashing, and the importance of all kinds of data being able to be stored in a hash table, was close to the hearts of the original designers of Java. Recall that every Java class has an ancestor named `Object` that contains various basic behaviors that are common to every object. One of the methods of the `Object` class is named `hashCode`. You can call `hashCode` on any object to convert the object into an integer for use in a hash table. A modified version of our hash function that works for any type of objects is the following:

```
public int hashFunction(Object value) {  
    return Math.abs(value.hashCode()) % elementData.length;  
}
```

According to the official Java documentation for the `hashCode` method, a proper hash function must be consistent. That is, when called multiple times on an object whose state is not changing, it should return the same value each time. Similarly, when `hashCode` is called on two objects with equivalent states (such that `equals` would return `true` for the two objects), it should return the same value.

In general it is also desirable for `hashCode` to return unequal values when two objects have different states. But it is not mandatory for unequal objects to always have different hash codes. In fact, it is impossible to guarantee such a thing; for example, there are only four billion unique integer values, but there are infinitely many unique strings, so some strings must naturally have the same hash code as others. Unequal objects having equal hash codes does not break the hash table, but it does mean that those objects will hash to the same index and will collide, so they will be placed into the same list if separate chaining is used on the hash table.

The default version of `hashCode` from the `Object` class computes an integer based on the memory address of the object. This function satisfies the consistency requirement, but it is not ideal because it does not take into account the state of each object. For this reason many classes override `hashCode` with their own versions that compute an integer by somehow combining the object's state. Good hash functions try to do their best about spreading the hash values over a wide range of numbers to minimize conflicts.

When writing your own class, if your class has an `equals` method, you are expected to also provide a `hashCode` method to override the one provided by the `Object` superclass. The reason is that objects with “equal” state are supposed to have equal hash codes, and this would not be guaranteed by the version of `hashCode` inherited from class `Object`. Your `hashCode` method should combine the object’s state in some way into a single integer and return it. The state can be combined in any way as long as it consistently produces the same integer for the same state.

The following is a possible `hashCode` method for the `Point` class implemented in [Chapter 8](#). The code adds the `x` and `y` values but scales the `y` value up by a large number to spread out the space of integers returned. This is to avoid collisions between points like (4, 7) and (7, 4). Simply adding `x + y` and returning the result would also be correct but would not distribute the integer codes as much.

```
// a possible hashCode method for Ch. 8's Point class
public class Point {
    private int x;
    private int y;
    ...
    public int hashCode() {
        return 31337 * y + x;
    }
}
```

Many classes in the Java class libraries have their own `hashCode` methods. For example, the `String` class code has a `hashCode` method whose code is roughly the following. The code seems odd in that it multiplies by 31 at each pass through the loop. If the characters were simply added, there would be conflicts between various kinds of strings. For example, strings of similar length would all be clustered within a small range of hash codes, and strings that are anagrams, such as “file” and “life”, would always collide. Multiplying the result by some number at each pass spreads out the space of codes and reduces these potential collisions.

```
// the hashCode method inside Java's String class
public int hashCode() {
    int hash = 0;
    for (int i = 0; i < this.length(); i++) {
        hash = 31 * hash + this.charAt(i);
    }
    return hash;
}
```

If you are writing a `hashCode` method for a class that has fields that are not integers, it is less clear how to combine the fields' state into a single hash code integer. Remember that if the fields are objects, they have their own `hashCode` methods that you can call. And primitives like `double` can be converted into their object wrapper equivalents, such as `Double`, and you can call `hashCode` on those.

Here is an example class `BankAccount` with non-integer fields and a possible `hashCode` method based on its state. Notice that we again multiply the various fields by large integer constants to spread out the results of the function.

```
// a possible hashCode method for a BankAccount class
public class BankAccount {
    private int id;
    private String name;
    private double balance;
    ...
    public int hashCode() {
        return id +
            31 * name.hashCode() +
            16581 * new Double(balance).hashCode();
    }
}
```

If we want our set to be able to store any kind of object as its data, we can convert our `HashSet` into a generic `HashSet`, giving it a type parameter `E` for its elements. This requires small changes throughout the code, such as changing methods to accept parameters of type `E` rather than `int`, and comparing values using `equals` rather than `==`. The process is very similar to our conversions of `ArrayList` and `LinkedList` to generic collections in previous chapters. The complete generic version of the class is not shown,

but a few of the relevant lines to change are found in the following abbreviated code:

```
public class HashSet<E> {  
    ...  
    public void add(E value) { ... }  
    public boolean contains(E value) {  
        ...  
        while (current != null) {  
            if (current.data.equals(value)) { ...  
        }  
        public void remove(E value) { ... }  
        ...  
        private class HashEntry {  
            private E data;  
            private HashEntry next;  
            ...  
        }  
    }  
}
```

Hash Map Implementation

The ideas shown so far form the core of the implementation of the `HashSet` class. The `HashMap` has a very similar internal structure.

Recall that a map stores key/value pairs rather than simple element

values. A `HashMap` is still implemented using a hash table, but the hash entry objects in each linked list are changed to store a pair of data fields, representing one key/value pair.

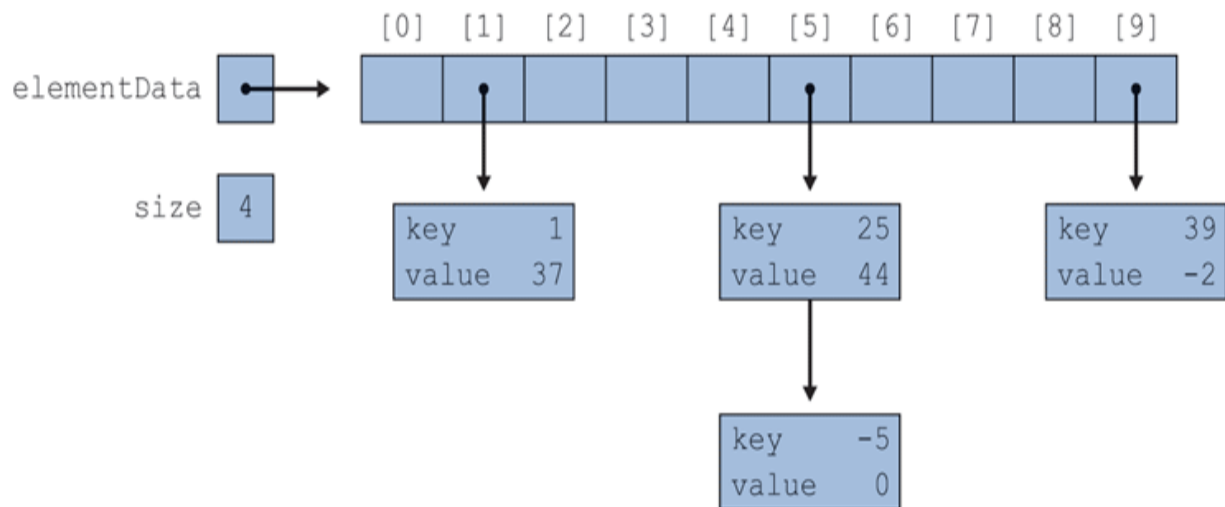
```
public class HashEntry {  
    private Object key;  
    private Object value;  
    private HashEntry next;  
    ...  
}
```

Suppose a map is created with integers as the type for its keys and values, and the following pairs are added to it:

```
mymap.put(1, 37);  
mymap.put(39, -2);  
mymap.put(25, 44);  
mymap.put(-5, 0);
```

Figure 18.12  shows the internal state of the hash table containing these pairs. Each index stores a list of `HashEntry` objects.

Figure 18.12 Hash map



To be able to accept any type of data for the keys and values, most hash map implementations are generic classes that accept a pair of type parameters, `K` for the key type and `V` for the value type. The hash map therefore has the following class header:

```
public class HashMap<K, V> {  
    ...  
    private class HashEntry {  
        private K key;  
        private V value;  
        private HashEntry next;  
        ...  
    }  
}
```

You can find complete code for the `HashSet` and `HashMap` classes on our web page at <http://buildingjavaprograms.com>.

18.2 Priority Queues and Heaps

In [Chapter 14](#), you saw how queues are useful for storing data in First-In, First-Out (FIFO) order. There are other situations in which a queue-like structure is needed but with an ordering other than FIFO. In this section we will discuss a collection called a priority queue that is useful in these situations. We will also explore an underlying data structure called a heap that can be used to efficiently implement a priority queue.

Priority Queues

Consider the task of simulating the waiting line of patients at an emergency room. In general it seems that the patients should be served in the order in which they arrived. But some patients require urgent care and should be able to step in front of others in line. The primary consideration in the line's ordering is the importance of treating that particular patient.

A similar situation is the task of processing the set of customers waiting to purchase their goods at a food market. Again the customers naturally want to gather into lines, but real markets have found that they can serve more customers if they have “express

lines” and other features to allow some customers to go ahead of others; in this case, customers with fewer items can skip ahead.

The general principle in situations like these is that of a queue, but organized by urgency or priority of each element rather than strictly on the order of insertion. There is a collection called a *priority queue* that is well suited to tasks like these. You can add elements to a priority queue in any order, but when you remove them, they will come out in order of priority or importance.

Priority Queue

A collection that associates each added element with a priority or importance and allows access or removal of the elements in order of priority.

Java’s Collections Framework in `java.util` includes a class called `PriorityQueue` that arranges elements into their natural ordering from smallest to largest. For primitive values like integers and real numbers this means arranging the elements in ascending order.

The following example briefly illustrates the usage of a priority queue. Seven integers are added to the queue in no particular order.

When the integers are removed from the queue, they come out in ascending order.

```
Queue<Integer> pq = new PriorityQueue<>();  
pq.add(42);  
pq.add(17);  
pq.add(9);  
pq.add(42);  
pq.add(35);  
pq.add(-1);  
pq.add(88);  
while (!pq.isEmpty()) {  
    System.out.print(pq.remove() + " "); // -1 9 17 35 42 42 88  
}
```

For objects, the priority queue arranges the elements by the ordering of their `compareTo` method. If the elements of the queue are not `Comparable` (or if you would like to arrange them in an order other than their natural ordering), you can pass a `Comparator` to the queue's constructor to indicate the relative ordering to use. The most commonly desired reordering is to reverse the queue so that the elements come out in descending rather than ascending order. This can be done easily by calling `Collections.reverseOrder`, a method that returns a `Comparator` that reverses the natural ordering of any comparable class:

```

Queue<Integer> pq = new PriorityQueue<>
(Collections.reverseOrder());
...
while (!pq.isEmpty()) {
    System.out.print(pq.remove() + " "); // 88 42 42 35 17 9 -1
}

```

The useful methods of `PriorityQueue` objects are listed in [Table 18.1](#) .

Table 18.1 `PriorityQueue<E>` Methods

A priority queue is a bit like a sorted list. You could achieve similar behavior to a priority queue by storing your data in an `ArrayList`, calling `Collections.sort` after adding each element, and always removing the element from index 0 every time. One key difference with a priority queue compared to a sorted list is that it manages the ordering automatically without the need for the client to sort the elements manually. Another big advantage is that the priority queue implements this behavior in a much more efficient way. Sorting a list is a slow operation that requires

$$O(N \log N)$$

time, and removing the first element from index 0 leads to bulky shifting of the elements, requiring

$$O(N)$$

time for each removal.

The priority queue, when implemented properly, can support its most common operations much more quickly than a sorted list. Adding and removing elements requires

$$O(\log N)$$

time, and peeking at the minimum element requires constant $O(1)$ time. The key to such an efficient implementation is a clever underlying data structure called a heap, which we will explore in the following sections.

Introduction to Heaps

A *heap* is a tree that arranges its elements with a vertical ordering. In a minimum heap or *min-heap*, child nodes always store values greater than or equal to the parent's value. In a maximum heap or *max-heap*, a parent node's value is always greater than or equal to those of its children. In other words, in a min-heap, the smallest element values are closer to the top (root) of the tree; in a max-heap, the largest element values are nearest to the root.

The fact that these elements are located at the top of the tree makes them easy to access and remove quickly and makes a heap an excellent structure to use to implement a priority queue. Since a priority queue is always finding and removing the smallest element,

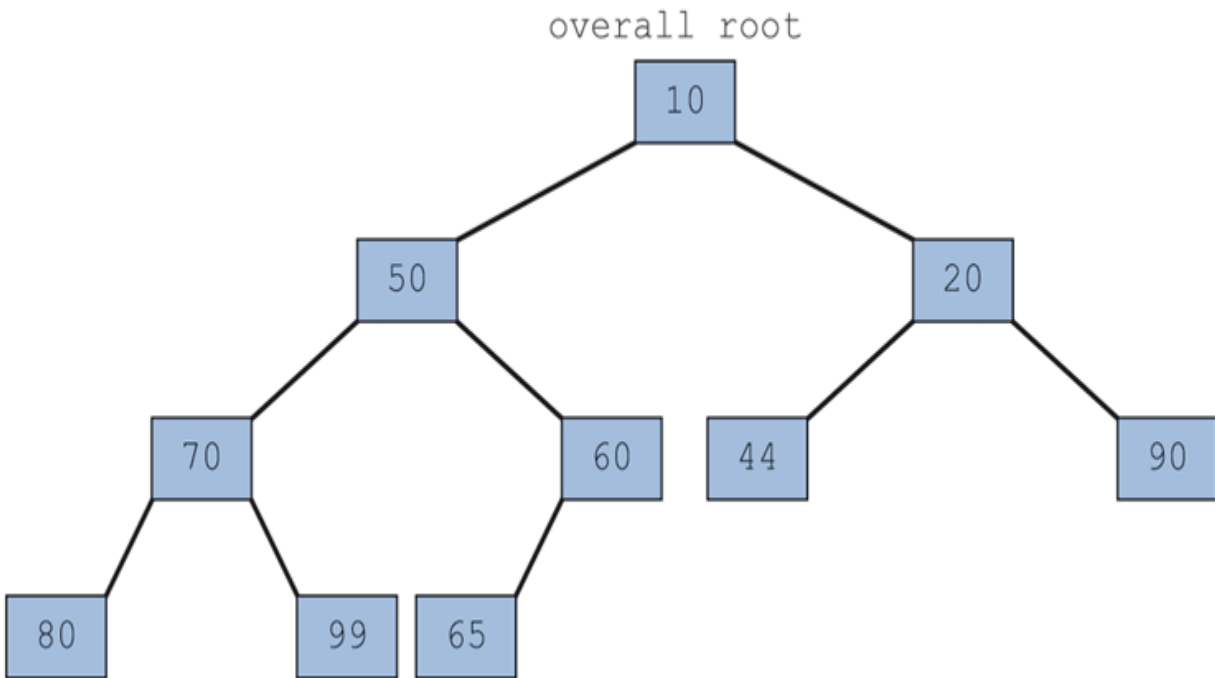
having that element at the very top of the tree makes finding it trivial. For this reason Java's `PriorityQueue` collection is implemented using a min-heap internally.

Heap

A tree that maintains a vertical ordering between parents and children, where parent nodes store smaller values than all of their children (min-heap) or parents store greater values than all of their children (max-heap).

Figure 18.13  shows an example of a min-heap. Notice how the smallest element value, `10`, is at the top of the tree. Larger elements are placed lower on the tree. You may be surprised initially by some of the ordering; `50` appears higher in the tree than `44`, and `90` is higher than `80` and `60`, and so on. But the key property of a heap is that a parent's value must be smaller than that of its children, and the nodes just listed are not in the same subtree as each other, so they share no direct parent–child relationship. Therefore, this example tree does satisfy the requirements to be a valid heap.

Figure 18.13 A min-heap



A heap can be implemented with nodes having any number of children, but as we did with trees in the previous chapter, we will focus on heaps whose nodes have at most two children, which are called *binary heaps*.

A heap is usually maintained as a *complete tree*, which is one where every level except possibly the lowest level is completely filled with all nodes having both children, and if the last level of the tree is not completely filled, all children are at the left-most edge of the tree. This minimizes the height of the tree relative to the number of elements; a wider and shorter tree is faster for traversing from the root to any given leaf.

Some people become confused about the difference between a priority queue and a heap. A priority queue is an abstract data type,

a description of a set of operations that a collection should have. A heap is one possible efficient way of implementing those operations, but it is not the only way of doing so. The Java class libraries add to this confusion by naming their class `PriorityQueue` when it should probably be represented as an interface named `PriorityQueue` that is implemented by a class called `HeapPriorityQueue`, to match other pairs like `List/ArrayList` or `Set/TreeSet`.

The key operations for a priority queue are adding elements and finding/removing the minimum value. Let's explore how those operations can be implemented efficiently using a heap. Finding the minimum value (`peek`) is trivial since it is simply the data stored in the overall root element of the tree. So we'll focus on adding and removing values from a heap.

Removing from a Heap

Suppose we have the heap shown in [Figure 18.13](#)  and want to remove the minimum value, 10. We can't simply yank the node out of the tree because it leaves a hole at the tree's root that must be filled. When dealing with binary search trees with their left-to-right ordering, removal usually involves moving one of the removed element's children up to occupy its spot and rearranging the tree as necessary to ensure that the ordering is left intact. With a heap we employ a different algorithm that begins not with the removed element's children but with a leaf.

To remove the root of a heap, first we remove the rightmost leaf of the tree. Then we place this leaf's value into the tree at the root, replacing the previous root value. This is likely to place the tree in a temporarily invalid state, since the leaf is presumably one of the larger element values and therefore is not suitable as a new root. But we then repair this problem by pushing the invalid root down the tree by swapping it with its smallest child until it is in a legal location, a process called *bubbling* or *percolating* the element down the tree.

Bubble (Percolate)

Moving a heap element upward or downward until the heap's state satisfies the required vertical ordering.

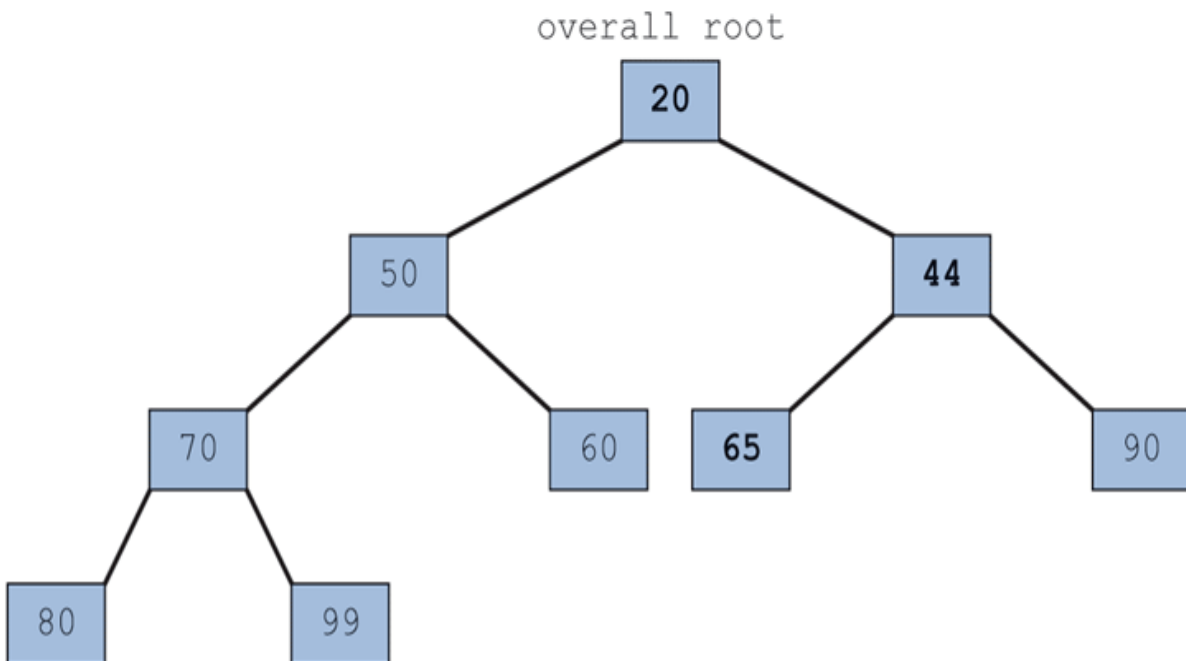
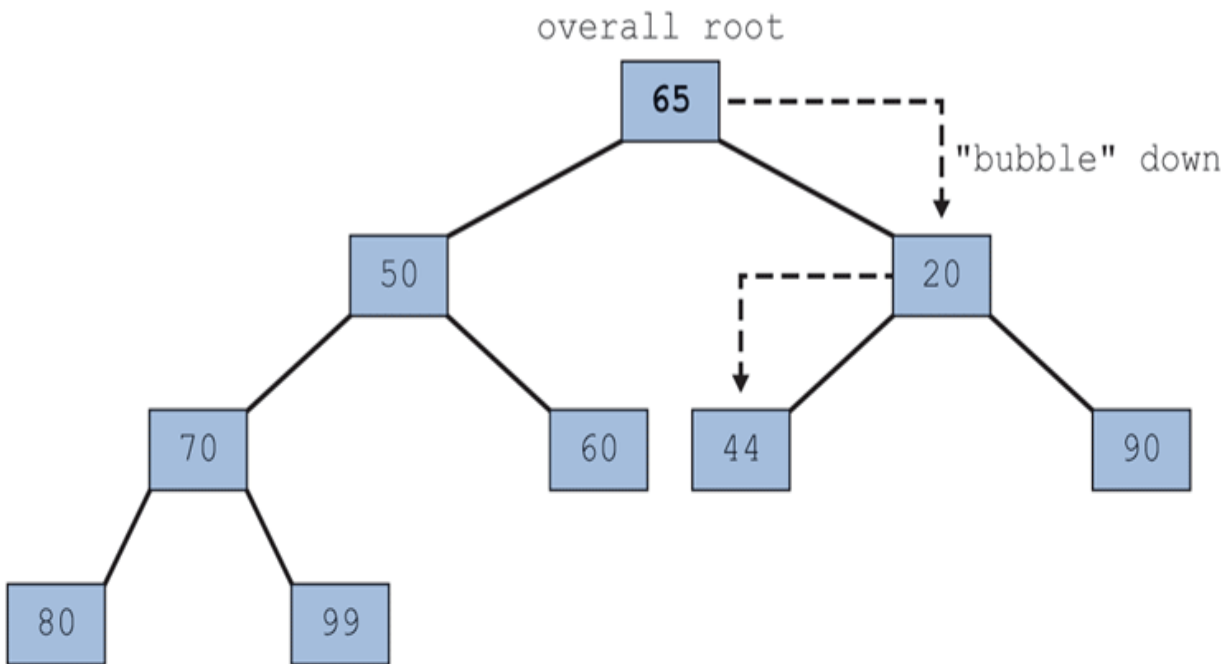
The overall remove-min algorithm is described by the following pseudocode:

```
find and remove rightmost leaf node.  
insert leaf's value into root node.  
current node = root.  
while (current node's value is not smaller than both children)  
{  
    choose smaller of two children.  
    swap current node's value with smaller child value.
```

```
current node = smaller child node.  
}
```

Figure 18.14  demonstrates the removal of the minimum value, 10, from our previous heap. First the rightmost leaf, 65, is inserted as the root. Then since 65 is not smaller than its children, it is swapped for its smaller child repeatedly until it is in proper order. The first swap is with 20, because $20 < 80$. Then 65 is compared with its new children, 44 and 90. The 44 is smaller so the 65 is swapped there. Now the heap is back into proper order so the removal is complete.

Figure 18.14 Removing from min-heap with bubble-down



The runtime of the remove-min operation is related to the number of swaps that occur as the new root is bubbled down to its proper location. Since our tree is a complete tree, it is as wide as possible for its height; the number of nodes in a tree of height H is roughly

equal to 2^H . Thus, the height of a tree containing N elements is very close to $\log_2 N$. This means that the number of bubble-down swaps, and therefore the overall runtime of removing the minimum element from a heap, is

$$O(\log N).$$

Adding to a Heap



Adding a value to a heap is similar to removing the minimum element but in reverse. Suppose we have the heap shown in [Figure 18.14](#) and want to add a new value, 37. The value needs to be placed properly to maintain the heap's overall ordering. The algorithm for adding an element to a heap begins at the bottom of the tree.

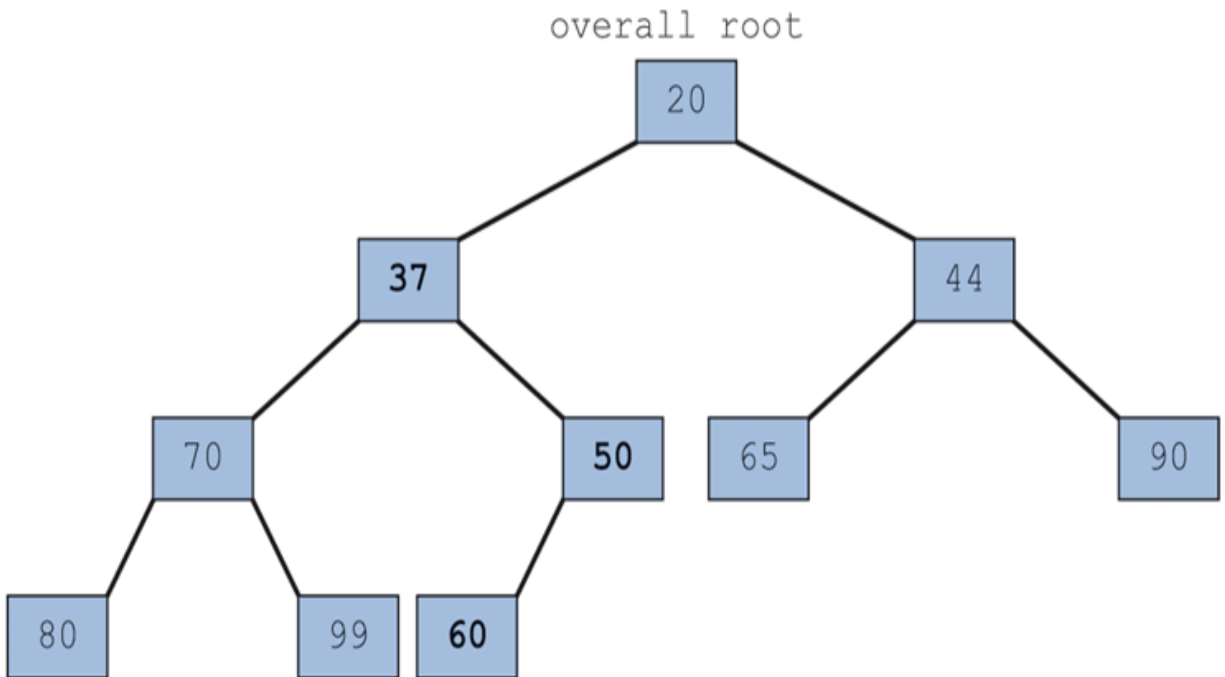
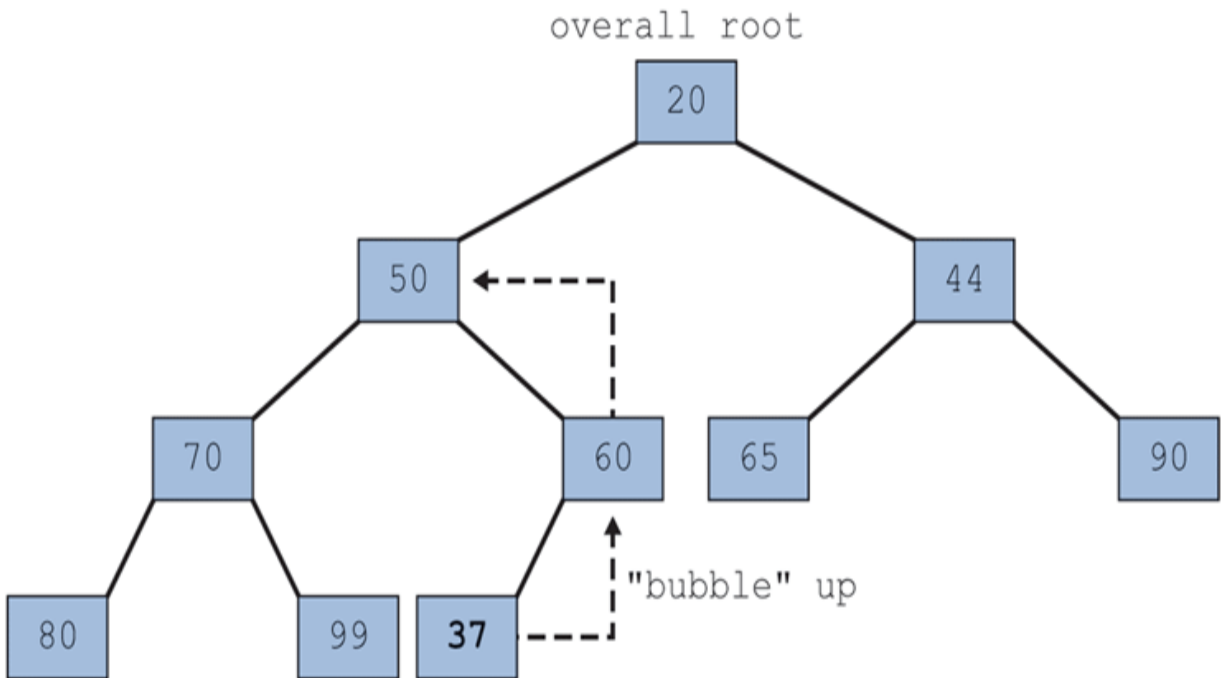
To add an element to a heap, we first insert it as a new rightmost leaf. (If the tree is completely full at its bottom level, the new value becomes the left child of the leftmost leaf, starting its own new level

in the tree.) This is likely to place the tree in a temporarily invalid state, since the newly inserted leaf may not be larger than all of its parents and therefore not suitable to be placed there. We repair this problem by swapping the new value upward with its parents until it is in a suitable location, bubbling it upward much like we bubbled values downward upon a removal. The overall pseudocode for adding to a heap is the following:

```
insert new element value as a new rightmost leaf node.  
current node = the new leaf node.  
while (current node's value is not greater than its parent) {  
    swap current node's value with parent's value.  
    current node = parent node.  
}
```

Figure 18.15  demonstrates adding the value 37 to our example heap from **Figure 18.14** . The value bubbles up twice, swapping with 60 and then with 50, because those values are greater than 37. It stops there because the next parent is 20, which is smaller than 37 and should therefore remain higher in the tree.

Figure 18.15 Adding to min-heap with bubble-up



The runtime complexity of adding to a heap is similar to that of removing the minimum element. In an ideal case, the newly added element will be larger than its parent and no upward bubbling will be

required, which would yield a constant $O(1)$ runtime. In the worst case, the newly added node is smaller than all of the values in the heap and must bubble all the way to the root. Since the height of a heap with N elements is almost exactly $\log_2 N$, the worst case for adding to the heap is

$$O(\log N).$$

In general the newly added element must bubble itself some portion of the distance up the tree, so we say that the `add` operation has a

$$O(\log N)$$

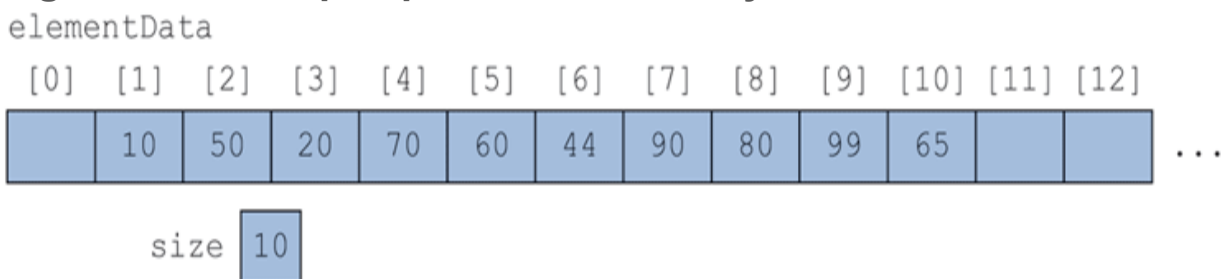
average runtime.

Array Heap Implementation

A heap can be implemented as a tree of linked node objects much like the binary trees implemented in the binary trees chapter. But a few of the operations we want to perform don't lend themselves well to the node implementation. For example, bubbling upward requires us to be able to reach a node's parent, and our tree implementation did not contain references from a node to its parent. Also, the add and remove operations both involve manipulating the rightmost leaf in the tree, which is not trivial to access in a tree implemented with nodes.

There is another clever implementation of heaps that provides great efficiency for the operations we want to implement. It turns out that any complete binary tree can actually be stored as an array, where index 1 stores the root element, and the children of index i are stored at indexes $2i$ and $2i + 1$. So the children of index 1 (the root) are at indexes 2 and 3; the children of index 2 are at indexes 4 and 5; the children of index 3 are indexes 6 and 7; the children of index 4 are indexes 8 and 9; the children of index 5 are indexes 10 and 11; and so on. We leave index 0 blank and start our root at index 1 so that the indexes line up conveniently in this way. **Figure 18.16** shows an array representation of the same heap depicted in **Figure 18.13**, with the value 10 as its root.

Figure 18.16 Heap implemented as array



The array representation is excellent for implementing a heap because it is easy to jump from a node to its parent; the parent of the element at index i is stored at index $i/2$. It is also trivial to find the rightmost child in the heap; the rightmost child is always located at index *size*, and any new rightmost child we want to insert should be inserted at index $size + 1$. For example, in the array heap shown in **Figure 18.16**, the rightmost leaf is 65, stored in index 10, and a new leaf would be added at index 11.

Let's implement a skeletal version of a priority queue using a heap represented as an array.

```
public class HeapIntPriorityQueue {  
    private int[] elementData;  
    private int size;  
  
    public HeapIntPriorityQueue() {  
        elementData = new int[10];  
        size = 0;  
    }  
    ...  
}
```

To simplify the implementation of the add and remove operations, we'll introduce several short helper methods for navigating upward and downward in the tree from parents to children and vice versa, as well as a method to swap elements in the array, since swapping is used in bubbling.

```
// helpers for navigating indexes up/down the tree  
private int parent(int index) {  
    return index / 2;  
}  
  
private int leftChild(int index) {  
    return index * 2;  
}
```

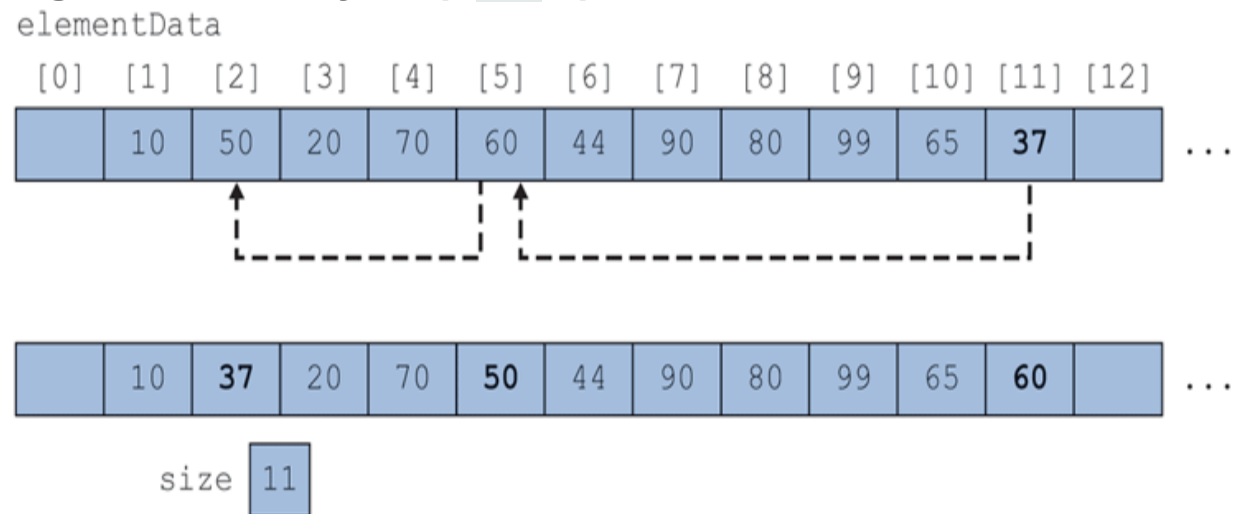
```

    }
    |
    |
    | private int rightChild(int index) {
    |     return index * 2 + 1;
    | }
    |
    |
    | private boolean hasParent(int index) {
    |     return index > 1;
    | }
    |
    |
    | private boolean hasLeftChild(int index) {
    |     return leftChild(index) <= size;
    | }
    |
    |
    | private boolean hasRightChild(int index) {
    |     return rightChild(index) <= size;
    | }
    |
    | private void swap(int[] a, int index1, int index2) {
    |     int temp = a[index1];
    |     a[index1] = a[index2];
    |     a[index2] = temp;
    | }

```

First let's implement the `add` operation, since it is a bit simpler than removal. **Figure 18.17**  shows the adding of element value `37` to the heap. It bubbles upward as far as necessary until the value is not smaller than its parent value or until it reaches the root of the tree.

Figure 18.17 Array heap `add` operation



The following code implements the `add` operation on our min-heap. The code uses the helper methods defined previously such as `parent` and `swap`. Notice that the node is initially inserted at the end of the array (index $size + 1$) and then bubbled upward to parent indexes as long as the node's value is less than that of its parent.

```
// Adds the given element to this queue.
public void add(int value) {
    elementData[size + 1] = value; // add as rightmost leaf
    // "bubble up" as necessary to fix ordering
    int index = size + 1;
    boolean found = false;
    while (!found && hasParent(index)) {
        int parent = parent(index);
        if (elementData[index] < elementData[parent]) {
```

```

        swap(elementData, index, parent(index));
        index = parent(index);
    } else {
        found = true; // found proper location; stop the
loop
    }
}

size++;
}

```

As with an array list or hash table, the array can run out of space. A proper `add` method should also check to make sure that the array has enough capacity to hold the element being added. The following code can be inserted at the start of the `add` method to handle this:

```

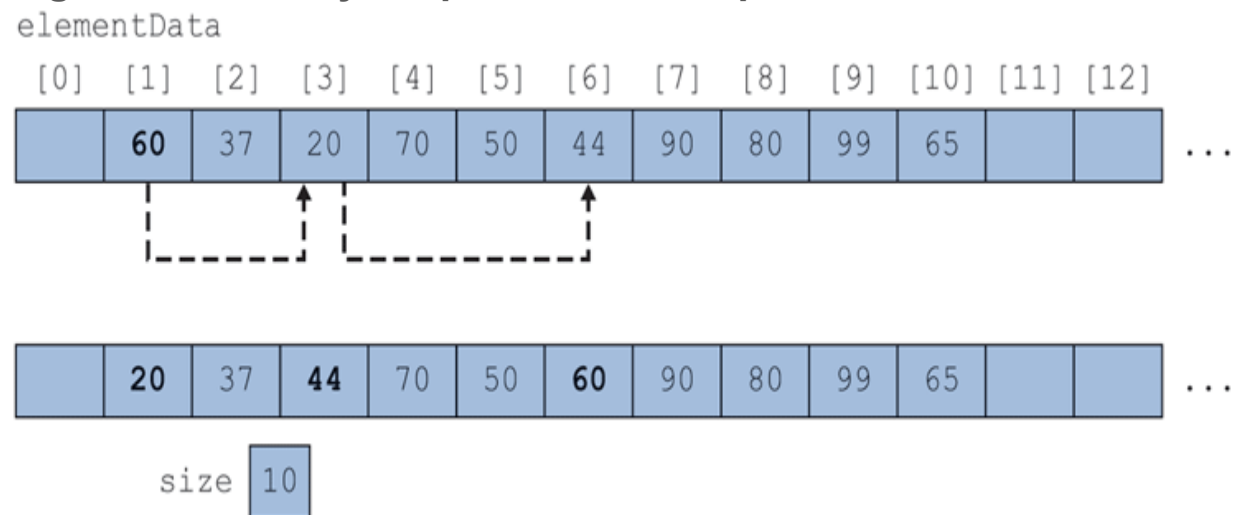
// resize if necessary
if (size + 1 >= elementData.length) {
    elementData = Arrays.copyOf(elementData, elementData.length
* 2);
}

```

Now let's implement the remove-min operation. **Figure 18.18**  shows the removal of the root element value from the heap from **Figure 18.17** . The rightmost leaf, `60`, is moved to the root, then it is bubbled downward, swapping it with its smaller child as far as

necessary until the value is not larger than either of its children (or until it is a leaf and has no children). In the example, 60 swaps with its right child, 20, which is smaller than 37; then it swaps with its left child, 44, which is less than its right child, 90.

Figure 18.18 Array heap remove-min operation



The following code implements the remove-min operation on our min-heap. The code uses the helper methods defined previously, such as `leftChild` and `hasRightChild`. The code inside the loop initially chooses to swap with the left child, but if the right one is smaller than the left, the right one is chosen. The loop stops when the node has no children (when it becomes a leaf) or when it is not larger than its child and has therefore reached the correct position.

```
// Removes and returns the minimum value in the queue.  
public int remove() {  
    // move rightmost leaf to become new root
```

```

    int result = peek();

    elementData[1] = elementData[size];

    size--;

    // "bubble down" as necessary to fix ordering

    int index = 1;

    boolean found = false;

    while (!found && hasLeftChild(index)) {

        int left = leftChild(index);

        int right = rightChild(index);

        int child = left;

        if (hasRightChild(index) &&
            elementData[right] < elementData[left]) {

            child = right;

        }

        if (elementData[index] > elementData[child]) {

            swap(elementData, index, child);

            index = child;

        } else {

            found = true; // found proper location; stop the
loop

        }

    }

    return result;
}

```

A heap that stores objects rather than integers is very similar to the code we have already shown, except that comparisons are made using the `compareTo` method rather than `<` and `>` operators. You can find complete code for the `HeapIntPriorityQueue` class, as well as a `HeapPriorityQueue` that stores objects, on our web page at <http://buildingjavaprograms.com>.

Heap Sort

In a previous chapter, we discussed details of several sorting algorithms, such as selection sort and merge sort. There is a simple algorithm for sorting called *heap sort* that takes advantage of the ordering of heaps to arrange data quickly. If you add all elements of an array to a priority queue and then remove them, they will come out in ascending (sorted) order. The heap sort algorithm can be described by the following pseudocode:

```
heapSort(A):  
    H = create new heap.  
    for each element n in A:  
        add n to H.  
    while (H not empty):  
        remove element from H.  
        add element back into A.
```

Here is a simple implementation of the heap sort for sorting an array of integers using a `PriorityQueue`:

```
public static void heapSort(int[] a) {  
    Queue<Integer> pq = new PriorityQueue<>();  
    for (int n : a) {  
        pq.add(n);  
    }  
    for (int i = 0; i < a.length; i++) {  
        a[i] = pq.remove();  
    }  
}
```

When sorting an array of N elements, heap sort performs N add and N remove operations on a heap. Each add or remove has a

$$O(\log N)$$

complexity, so the overall heap sort algorithm has

$$O(N \log N)$$

complexity. Heap sort is a fairly efficient algorithm, certainly much faster than selection sort and with comparable performance to merge sort.

One drawback of the heap sort algorithm as shown is the memory required. To sort the array we must create another large data

structure (the priority queue, and the array heap inside it) to temporarily store the data. For very large data sets this can be undesirable.

It is possible to implement heap sort “in-place” so that it uses the original array itself as the heap. The basic idea is to treat the array as an initially invalid heap and “repair” it into a proper heap by repeatedly bubbling elements into proper positions. Once the array represents a heap, performing `remove` operations repeatedly on the array and moving the removed elements to the end of the array end up fully sorting the array contents. The algorithm arranges the elements into ascending order if a max-heap is used and descending order if a min-heap such as our `HeapIntPriorityQueue` is used.

The in-place version of the heap sort algorithm would require a modified version of our `HeapIntPriorityQueue` implementation with a few tweaks. The details of implementing an in-place heap sort algorithm are outside the scope of this chapter, but a full implementation of the algorithm is posted on our web page at <http://buildingjavaprograms.com>.

Chapter Summary

Hashing is a technique for mapping between element values and integers representing array indexes at which to store those elements. Hashing allows for the implementation of sets and maps with extremely efficient $O(1)$ runtime for adding, removing, and searching for elements.

A hash function is an algorithm that maps element values to array indexes. A common technique for integer data is to mod (%) the value by the array's length.

When multiple element values have the same hash function value, there is a collision. Collisions can be resolved by moving to another index (probing) or by storing a list of matched elements at each index (separate chaining).

Rehashing is the process of resizing a hash table, generally to enlarge it so that it can efficiently store more elements. The array cannot simply be enlarged; the elements must be readded because their hash function values may change in the new larger array.

Non-integer elements can be hashed by calling their `hashCode` method.

A priority queue allows elements to be inserted in any order and removed in ascending order.

A structure called a heap is often used to implement priority queues. A heap is a tree with a vertical ordering, where parents store smaller values than their children (min-heap) or larger values than their children (max-heap).

When adding a value to a heap, it is added as a leaf and then “bubbled” (swapped) up the tree until it is in a proper place in the vertical ordering. When removing a value from a heap, the heap’s rightmost leaf is moved to the root and then “bubbled” down the tree until it is in a proper place in the vertical ordering.

Because of its complete structure, a heap can be implemented efficiently using an array, rather than as a linked tree of node objects.

Heaps can be used to help sort data, an algorithm called heap sort.

Self-Check Problems

Section 18.1: Hashing

1. What is hashing, and why is hashing a good way of implementing a set?
2. Which of the following statements about hash tables is true?
 - a. A hash table has

$$O(\log N)$$

average time to add and search for elements.

- b. The higher a hash table's load factor, the more quickly elements can be found.
 - c. Once a hash table's load factor reaches 0.75, no more elements can be added.
 - d. A hash function maps element values to integer indexes in the hash table.
 - e. A good hash function returns the same value as much as possible for all data.
3. How do you know when a hash table has become "full"? At what point does it become impossible to add further elements to a hash table of a given size? Does it depend on the collision resolution strategy used?
4. The following statement is an incorrect attempt to resize a hash table to twice its current size. Why is this code incorrect? What is the proper way to enlarge a hash table?

```
elementData = Arrays.copyOf(elementData, 2 *  
elementData.length);
```

5. What is the final state of a hash table of size 10 after adding 35, 2, 15, 80, 42, 95, and 66? Assume that we are using the standard “mod” hash function shown in the chapter and linear probing for collision resolution. Do not perform any resizing or rehashing. Draw the entire array and the contents of each index.
6. If separate chaining is used for collision resolution, and the same elements from the previous problem (35, 2, 15, 80, 42, 95, and 66) are added to a hash table of size 10, what is the final state of the hash table? Do not perform any resizing or rehashing. Draw the entire array and the contents of each index.
7. Suppose we have a hash set that uses the standard “mod” hash function shown in the chapter and uses linear probing for collision resolution. The starting hash table length is 5, and the table chooses to rehash to twice its former size once the load factor reaches or exceeds 0.5. If we begin with an empty set, what will be the final state of the hash table after the following elements are added and removed? Draw the entire array and the contents of each index, including any resizing and rehashing necessary. Write “X” in any index in which an element is removed and not replaced by another element. Also write the size, capacity, and load factor of the final hash table.

```
HashIntSet set = new HashIntSet();  
set.add(15);  
set.add(5);  
set.add(13);  
set.add(24);  
set.add(32);  
set.remove(13);  
set.add(17);  
set.add(44);  
set.remove(15);  
set.add(47);
```

8. Suppose we have a hash set that uses the standard “mod” hash function shown in the chapter and uses linear probing for collision resolution. The starting hash table length is 11, and the table does not rehash during this problem. If we begin with an empty set, what will be the final state of the hash table after the following elements are added and removed? Draw the entire array and the contents of each index. Write “X” in any index in which an element is removed and not replaced by another element. Also write the size, capacity, and load factor of the final hash table.

```
set.add(4);  
set.add(52);  
set.add(50);  
set.add(39);
```

```
set.add(29);  
set.remove(4);  
set.remove(52);  
set.add(70);  
set.add(82);  
set.add(15);  
set.add(18);
```

9. For each of the following possible `hashCode` implementations:
Is the following a legal `hashCode` method for a `Point` class,
according to the general contract of that method? Does it
distribute the hash codes well between objects? Why or why
not?

a.

```
public int hashCode() {  
    return x * y;  
}
```

b.

```
public int hashCode() {  
    return 42;  
}
```

C.

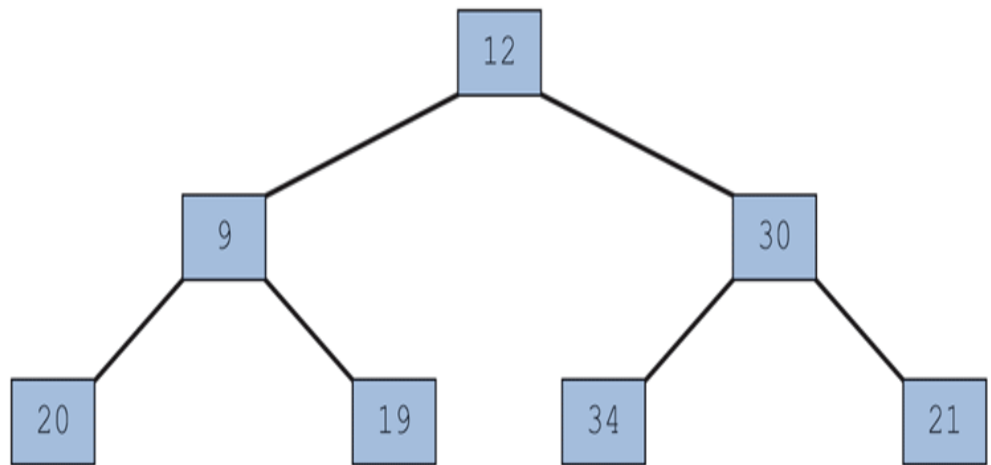
```
public int hashCode() {  
    Random rand = new Random();  
    return rand.nextInt();  
}
```

10. Write a `hashCode` method for a `Date` class, whose fields are a year, month, and day, as integers. Follow the general contract for the `hashCode` method.
11. Write a `hashCode` method for a `Student` class, whose fields are a name (a string), age (an integer), student ID number (an integer), and weight in pounds (a real number). Follow the general contract for the `hashCode` method.
12. Suppose we have a hash map that uses the standard “mod” hash function shown in the chapter and uses linear probing for collision resolution. The starting hash table length is 5, and the table chooses to rehash to twice its former size once the load factor reaches or exceeds 0.5. If we begin with an empty map, what will be the final state of the hash table after the following key/value pairs are added and removed? Draw the entire array and the contents of each index, including any resizing and rehashing necessary. Write “X” in any index in which an element is removed and not replaced by another element. Also write the size, capacity, and load factor of the final hash table.

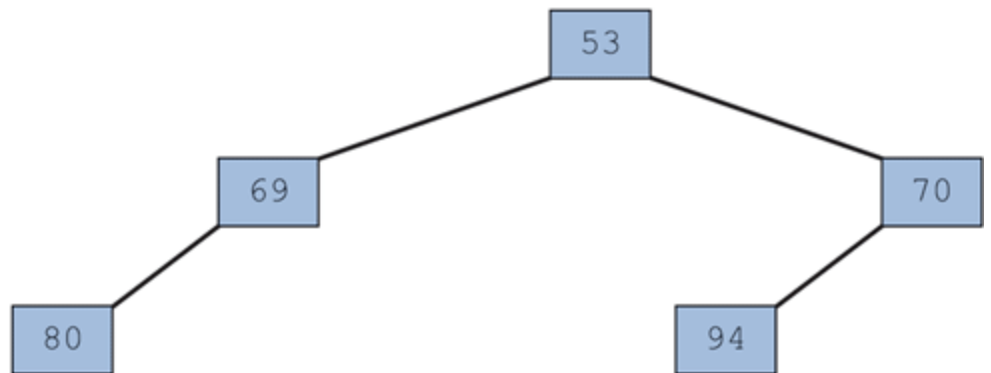
```
HashMap<Integer, String> map = new HashMap<>();  
map.put(7, "Jessica");  
map.put(34, "Tyler");  
map.put(17, "Ryan");  
map.put(15, "Tina");  
map.put(84, "Saptarshi");  
map.remove("Tyler");  
map.put(7, "Meghan");  
map.put(33, "Kona");  
map.remove(17);  
map.put(6, "Tina");  
map.remove(84);  
map.put(15, "Daisy");
```

Section 18.2: Priority Queues and Heaps

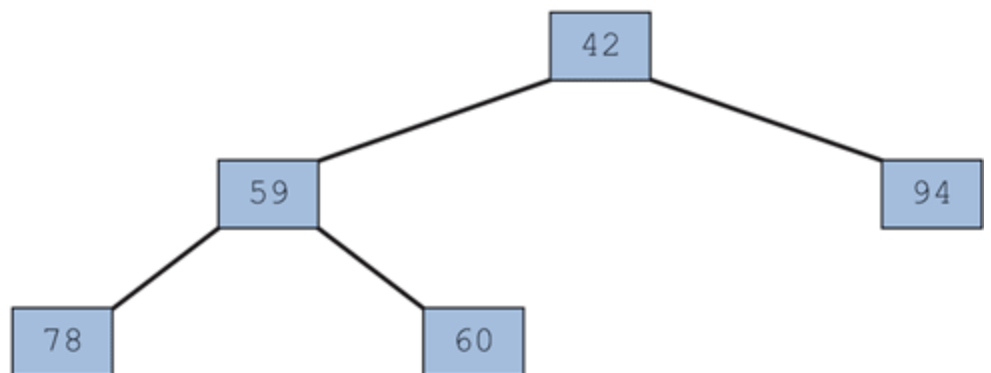
13. Which of the following statements about min-heaps is true?
- a. Smaller values are on the left and larger values are on the right.
 - b. The smallest value is the root.
 - c. The smallest value is one of the leaves.
 - d. Every value is smaller than all values at lower levels of the tree. For example, if there is a 25 at level 3, there will not be any elements with values less than 25 at levels 4 and beyond.
 - e. The peek operation returns the largest value in the heap.
14. If a binary heap has 26 nodes, what is its height? If it has 73 nodes, what is the height? How do you know for sure?
15. Which of the following are valid min-heaps? For the one(s) that are invalid, what makes them invalid?
- a. 



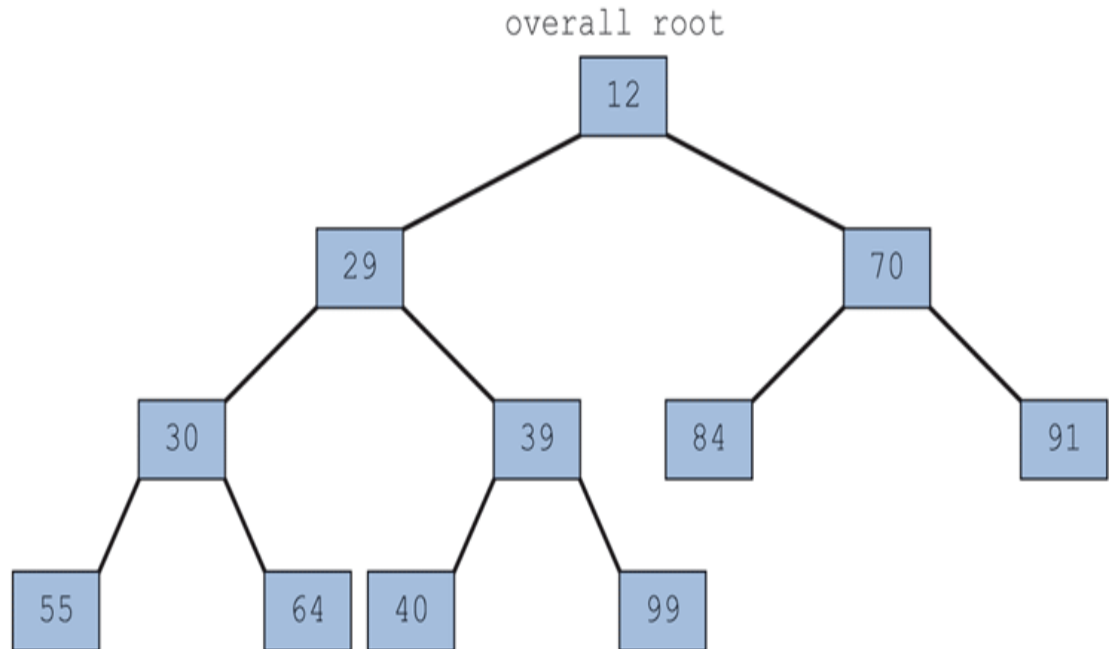
b.



c.



16. In an array heap as implemented in this section, for the element at index 8 of the array, what are the indexes of its left and right children? What is the index of its parent? If the element is at index 23, what are the indexes of its children and parent?
17. Simulate the adding of the value 21 to the following min-heap:



18. Simulate the adding of value 7 to the same heap from the previous problem, after the 21 has already been added.
19. Draw the tree for the binary min-heap that results from inserting 4, 9, 3, 7, 2, 5, 8, 6 in that order into an initially empty heap.
20. Perform 3 removals on the heap you drew in the previous problem. Show the complete state of the tree after each removal.

21. Draw the tree for the binary min-heap that results from inserting 11, 9, 12, 14, 3, 15, 7, 8, 1 in that order into an initially empty heap.
22. Perform 3 removals on the heap you drew in the previous problem. Show the complete state of the tree after each removal.
23. The following is an incorrect diagram of the array representations of the heap shown in Self-Check Problem 17 using the array representation described in this section of the chapter (before any adds or removals are made from it). What is wrong with the diagram? What changes must be made to fix it so that it accurately represents the given heap?

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	
12	29	30	39	40	55	64	70	84	91	99			...

24. Draw the min-heap being represented by the given array:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	29	41	30	55	68	37	41	80	

25. Draw the array representation of the heap you computed as your answer to Self-Check Problem 19 (after all of the elements are added to it).
26. Draw the array representation of the heap you computed as your answer to Self-Check Problem 21 (after all of the

elements are added to it).

Exercises

1. Write a method in the `HashIntSet` class called `addAll` that accepts another hash set as a parameter and adds all of the elements from the other set into the current set. For example, if the set stores `[-5, 1, 2, 3]` and the method is passed `[2, 3, 6, 44, 79]`, your set would store `[-5, 1, 2, 3, 6, 44, 79]`.
2. Write a method in the `HashIntSet` class called `containsAll` that accepts another hash set as a parameter and returns `true` if your set contains every element from the other set. For example, if the set stores `[-2, 3, 5, 6, 8]` and the method is passed `[3, 6, 8]`, your method would return `true`. If the method were passed `[3, 6, 7, 8]`, your method would return `false` because your set does not contain the value `7`.
3. Write a method in the `HashIntSet` class called `equals` that accepts another hash set as a parameter and returns `true` if the two sets contain exactly the same elements. The internal hash table size and ordering of the elements does not matter, only the sets of elements themselves.
4. Write a method in the `HashIntSet` class called `removeAll` that accepts another hash set as a parameter and ensures that this set does not contain any of the elements from the other set. For example, if the set stores `[-2, 3, 5, 6, 8]` and the

method is passed `[2, 3, 6, 8, 11]`, your set would store `[-2, 5]`.

5. Write a method in the `HashIntSet` class called `retainAll` that accepts another hash set as a parameter and removes all elements from this set that are not contained in the other set. For example, if the set stores `[-2, 3, 5, 6, 8]` and the method is passed `[2, 3, 6, 8, 11]`, your set would store `[3, 6, 8]`.
6. Write a method in the `HashIntSet` class called `toArray` that returns the elements of the set as a filled array. The order of the elements in the array is not important as long as all elements from the set are present in the array, with no extra empty slots before or afterward.
7. Write a method in the `HashIntSet` class called `toString` that returns a string representation of the elements in the set, such as `"[-2, 3, 5, 6, 8]"`. The order of the elements in the string does not matter as long as they are all present in the proper format. Do not list any empty or meaningless indexes in the string.
8. Write a method called `descending` that accepts an array of integers and rearranges the integers in the array to be in descending order using a `PriorityQueue` as a helper. For example, if the array passed stores `[42, 9, 22, 17, -3, 81]`, after the call the array should store `[81, 42, 22, 17, 9, -3]`.
9. Write a method called `kthSmallest` that accepts a `PriorityQueue` of integers and an integer k as parameters and

returns the k th-smallest integer from the priority queue. For example, if the queue passed stores the integers `[42, 50, 45, 78, 61]` and k is 4, return the fourth-smallest integer, which is `61`. If k is 0 or negative or greater than the size of the queue, throw an `IllegalArgumentException`. If your method modifies the state of the queue during its computation, it should restore the queue before it returns. You may use one stack or queue as auxiliary storage.

10. Write a method called `isConsecutive` that accepts a `PriorityQueue` of integers as a parameter and returns `true` if the queue contains a sequence of consecutive integers starting from the front of the queue. Consecutive integers are integers that come one after the other, as in 5, 6, 7, 8, 9, etc., so if the queue stores `[7, 8, 9, 10, 11]`, your method should return `true`. If your method modifies the state of the queue during its computation, it should restore the queue before it returns. You may use one stack or queue as auxiliary storage.
11. Write a method called `removeDuplicates` that accepts a `PriorityQueue` of integers as a parameter and modifies the queue's state so that any element that is equal to another element in the queue is removed. For example, if the queue stores `[7, 7, 8, 8, 8, 10, 45, 45]`, your method should modify the queue to store `[7, 8, 10, 45]`. You may use one stack or queue as auxiliary storage.
12. Write a method called `stutter` that accepts a `PriorityQueue` of integers as a parameter and replaces every value in the queue with two occurrences of that value. For example, if the

queue stores `[7, 8, 10, 45]`, your method should modify the queue to store `[7, 7, 8, 8, 10, 10, 45, 45]`. You may use one stack or queue as auxiliary storage.

13. Write a method called `fillGaps` that accepts a `PriorityQueue` of integers as a parameter and adds elements to it until every element in its range of smallest to largest is represented once if it was not previously found in the priority queue. For example, if the queue stores `[1, 3, 3, 6, 6, 8]`, your method should modify the queue to store `[1, 2, 3, 3, 4, 5, 6, 6, 7, 8]`. You may use one collection as auxiliary storage.
14. Write a method in the `HeapIntPriorityQueue` class called `toArray` that returns the elements of the queue as a filled array. The order of the elements in the array is not important as long as all elements from the queue are present in the array, with no extra empty slots before or afterward.
15. Write a method in the `HeapIntPriorityQueue` class called `toString` that returns a string representation of the elements in the queue, such as `"[42, 50, 45, 78, 61]"`. The order of the elements in the string does not matter as long as they are all present in the proper format.
16. Write a method in the `HeapIntPriorityQueue` class called `merge` that accepts another `HeapIntPriorityQueue` as a parameter and adds all elements from the other queue into the current queue, maintaining proper heap order such that the elements will still come out in ascending order when they are removed. Your code should not modify the queue passed in as a

parameter. (Recall that objects of the same class can access each other's private fields.)

Programming Projects

1. Add an iterator to the `HashSet` class written in this chapter. To do this you will need to write an inner class that can iterate over the elements of the set, remembering its position as it moves along. Also write a method in the `HashSet` class called `iterator` that returns such an iterator object.
2. In this chapter, we implemented a `HashSet` that used separate chaining to resolve collisions. Implement your own version of `HashSet` that uses linear probing to resolve collisions. Perform lazy removals, placing a special “removed” marker value into any bucket where an element is removed.
3. Modify the `HeapIntPriorityQueue` class written in this chapter to make it configurable in ways similar to Java’s `PriorityQueue` class. Make it possible for the heap to be a min-heap or max-heap. (If you create a heap of objects, you could also modify it to accept a `Comparator` parameter to its constructor.)
4. Modify the `HeapIntPriorityQueue` class written in this chapter to make it into a three-heap. A three-heap is similar to a binary heap, except that each node is considered to have three children rather than two children. A three-heap is a wider tree though not as tall as a binary heap. The heap can still be represented as an array, but the algorithms for traversing it must be modified to consider all three children when bubbling as appropriate, etc. (Do you expect that a

three-heap will be faster or slower than a binary heap for insertion, and for removal? Why? You can create a test to verify the performance of each operation.)

5. Write a program that implements the “Huffman coding” compression algorithm using priority queues and binary trees. Huffman coding is an algorithm devised by David A. Huffman of MIT in 1952 for compressing text data to make a file occupy a smaller number of bytes. Normally text data is stored in a standard format of 8 bits per character, commonly using an encoding called ASCII that maps every character to a binary integer value from 0–255. The idea of Huffman coding is to abandon the rigid 8-bits-per-character requirement and use different-length binary encodings for different characters. The advantage of doing this is that if a character occurs frequently in the file, such as the letter “e”, it could be given a shorter encoding (fewer bits), making the file smaller.

The steps involved in Huffman coding a given text source file into a destination compressed file are the following:

- a. Examine the source file’s contents and count the number of occurrences of each character (consider using a map).
- b. Place each character and its frequency (count of occurrences) into a priority queue ordered in ascending order by character frequency.
- c. Convert the contents of this priority queue into a binary tree with a particular structure. Create this tree by repeatedly removing the two front elements from the

priority queue (the two nodes with the lowest frequencies) and combining them into a new node with these two nodes as its children and the two nodes' combined frequencies as its frequency. Then reinsert this combined node back into the priority queue. Repeat until the priority queue contains just one single node.

- d. Traverse the tree to discover the binary encodings of each character. Each left branch represents a '0' in the character's encoding and each right branch represents a "1".
- e. Reexamine the source file's contents, and for each character, output the encoded binary version of that character to the destination file to compress it.